

TrojanGPL: Vulnerability Analysis and Mitigation of Backdoor Attacks on Graph Prompt Learning

Xiaoting Lyu^{ID}, Yufei Han^{ID}, Hangwei Qian^{ID}, Ivor W. Tsang^{ID}, *Fellow, IEEE*, and Wei Wang^{ID}

Abstract—Graph Prompt Learning (GPL) bridges graph pre-training and downstream tasks, enabling effective knowledge transfer. However, its vulnerability to backdoor attacks during pre-training remains largely unexplored. This article presents a systematic study of backdoor risks in GPL and proposes *TrojanGPL*, a cross-context backdoor attack that injects triggers solely by manipulating the pre-trained encoder, without requiring downstream task knowledge. We show that jointly optimizing trigger graphs and the prompt mechanism aligns upstream and downstream objectives, as well as main-task and backdoor learning losses, thereby enabling reliable backdoor transfer across downstream applications. To mitigate this threat, we introduce *ES-Detector*, a defense approach that detects backdoored graphs by identifying discriminative embedding shifts caused by the removal of important nodes. When clean and backdoored graphs are classified identically, clean graphs exhibit stable embedding changes, whereas backdoored graphs show significant deviations. Experiments on five benchmark datasets and two GPL frameworks demonstrate that *TrojanGPL* achieves attack success rates above 90% in diverse cross-context settings with negligible impact on main-task performance. Meanwhile, *ES-Detector* reduces attack success rates by at least 60% across all settings, demonstrating strong robustness without prior knowledge of attack patterns.

Index Terms—Graph prompt learning, backdoor attacks.

I. INTRODUCTION

GRAPH Prompt Learning (GPL) [1], [2], [3], [4], [5] aims to reduce the generalization gap between pretrained graph neural networks (GNNs) and diverse downstream applications. In practice, pretraining and deployment often occur under different conditions. For example, a GNN encoder pretrained on

social network data may later be adapted for recommendation systems, or a model trained for link prediction may be reused for node or graph classification. Such shifts introduce discrepancies in semantic spaces, structural patterns, and learning objectives, which hinder effective knowledge transfer.

To address this issue, GPL adopts a prompting paradigm that reformulates downstream tasks to better align with pretrained representations, thereby reducing the reliance on extensive fine-tuning. By incorporating task-specific prompts, GPL enables efficient adaptation of pretrained GNN encoders with limited labeled data, improving transferability and data efficiency in few-shot graph learning scenarios.

Despite these benefits, the pretraining–prompting paradigm also introduces security risks, making GPL susceptible to backdoor attacks. In this setting, attackers implant triggers into the GNN encoder during pretraining, which remain dormant on benign inputs but activate on specific trigger patterns during downstream inference, leading to targeted misclassifications. This threat is particularly concerning in practice, as pretrained GNN encoders are often obtained from untrusted third-party sources without access to training data or implementation details. Once a compromised backdoor encoder is incorporated into a GPL pipeline, backdoor behavior can propagate transparently to downstream applications.

Although prior work [6], [7], [8], [9], [10], [11] has demonstrated that GNNs are vulnerable to backdoor attacks under supervised training, these findings do not directly extend to GPL. In GPL, attackers are restricted to manipulating unlabeled pretraining data and have no access to downstream task specifications, while substantial distribution shifts between the pretraining and downstream stages further hinder backdoor transferability. At the same time, effective attacks must preserve benign performance and remain stealthy under prompt-based adaptation, imposing additional constraints that are absent in supervised settings. As a result, the backdoor vulnerability of GPL remains largely unexplored, both in terms of attack feasibility and principled defense design. To fill this gap, this work conducts a systematic investigation of backdoor risks in GPL, examining how prompt learning affects both attack transferability and defense effectiveness, and providing practical insights into securing GPL-based systems.

Our Work. This work presents a systematic study of backdoor vulnerabilities in GPL, covering both attack surfaces and defense mechanisms. We organize our analysis around three research questions.

Received 10 April 2025; revised 26 May 2026; accepted 2 June 2026. Date of publication 10 June 2026; date of current version 1 September 2026. This work was supported in part by the Open Funding Programs of the State Key Laboratory of AI Safety under Grant 2025-03, in part by the China Postdoctoral Science Foundation under Grant 2025M781515, in part by the Shaanxi Provincial Postdoctoral Research Funding Project, in part by the National Natural Science Foundation of China under Grant 62441227 and Grant U22B2027, and in part by the Hangzhou Qianjiang Distinguished Experts Programme. (*Corresponding author: Wei Wang.*)

Xiaoting Lyu is with the Ministry of Education Key Lab for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China (e-mail: xiaoting.lyu@xjtu.edu.cn).

Yufei Han is with INRIA, 35042 Rennes, France (e-mail: yufei.han@inria.fr). Hangwei Qian and Ivor W. Tsang are with CFAR, A*STAR, Singapore 138632 (e-mail: qian_hangwei@cfar.a-star.edu.sg; ivor_tsang@cfar.a-star.edu.sg).

Wei Wang is with the Ministry of Education Key Lab for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China, and also with the Beijing Jiaotong University, Beijing 100044, China (e-mail: wei.wang@xjtu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3702151

RQ1: What inherent vulnerabilities in GPL systems make them susceptible to backdoor attacks?

We first analyze the security risks of GPL under backdoor threats. Our theoretical analysis reveals that, although GPL facilitates knowledge transfer across diverse downstream graph learning tasks, it also introduces inherent vulnerabilities that enable backdoor transfer. Specifically, GPL reuses pretrained GNN encoders and relies on prompt-based mechanisms to bridge pretraining and downstream objectives, thereby supporting efficient few-shot adaptation. However, this design unintentionally allows backdoor artifacts embedded in the pretrained encoder to propagate into downstream applications. Moreover, we show that carefully constructed trigger graphs that align the backdoor objective with the main task objective can implant persistent backdoor effects into the GNN encoder. These backdoors remain effective even after downstream prompt-based adaptation, highlighting a fundamental security weakness in the GPL paradigm.

RQ2: How can effective backdoor attacks be designed for GPL with different cross-domain scenarios without requiring knowledge of downstream tasks?

Based on previous findings, we introduce *TrojanGPL*, a novel cross-context backdoor attack framework that optimizes trigger graphs to align the backdoor learning task with the encoder's pretraining learning task. We formulate *TrojanGPL* as a feature collision optimization problem, where the embeddings of backdoored graphs are manipulated to align with those of trigger graphs while remaining distinguishable from clean samples within the pretraining dataset. By preserving the semantic integrity of clean samples and ensuring the seamless transferability of backdoor effects, *TrojanGPL* effectively exploits the intrinsic alignment properties of GPL.

RQ3: How can such transferable and knowledge-agnostic backdoor attacks be effectively detected and mitigated without tuning the GNN encoder?

To mitigate backdoors embedded in pretrained GNN encoders during downstream deployment, we propose *ES-Detector*, a defense method based on embedding-shift consistency analysis. *ES-Detector* is motivated by the observation that, for clean graphs belonging to the same class, removing important nodes induces stable and predictable embedding shifts, whereas backdoored graphs exhibit pronounced and irregular deviations. By analyzing embedding variations caused by targeted node removal and comparing them with those observed on clean reference graphs, *ES-Detector* can effectively identify anomalous behaviors indicative of backdoor triggers.

We conduct extensive experiments on two representative GPL frameworks (ProG and GraphPrompt) across five benchmark datasets, covering five cross-context scenarios: cross-distribution, cross-class, cross-dataset, cross-domain, and cross-task. The results show that *TrojanGPL* consistently achieves attack success rates above 0.85 with minimal impact on benign performance, incurring accuracy drops of less than 0.06 across all evaluated settings. Besides, our proposed defense, *ES-Detector*, reduces attack success rates by 0.60 to 1.00 in all scenarios without requiring prior knowledge of attack patterns. These results highlight both the severity of backdoor threats in

GPL and the effectiveness of *ES-Detector* in securing diverse GPL applications.

This paper is a substantially extended version of our preliminary work [12]. First, we substantially revise and extend the theoretical analysis to provide a clearer and more systematic characterization of the attack mechanism, explicitly linking embedding alignment and loss coupling to the design of *TrojanGPL*. **Second**, we broaden the scope of the study by conducting a comprehensive analysis of defense mechanisms in GPL, demonstrating that representative defenses are largely ineffective against *TrojanGPL*. **Third**, we newly propose a defense method, *ES-Detector*, and provide a theoretical analysis to formally justify its effectiveness. Specifically, we analyze why clean graphs exhibit consistent embedding shifts after important-node removal, whereas backdoored graphs show abnormal deviations due to trigger-induced perturbations. This extension introduces new algorithmic designs, theoretical analysis, and experimental validation beyond the preliminary work.

In summary, our main contributions are as follows:

- We propose *TrojanGPL*, a backdoor attack against GPL systems that exploits the prompt mechanism and the trigger optimization to implant transferable backdoors.
- We introduce *ES-Detector*, a defense method that exploits embedding-shift consistency under controlled node removal to identify and mitigate backdoor triggers.
- Through extensive evaluations across multiple benchmark datasets and diverse application scenarios, we demonstrate both the severity of the threat posed by *TrojanGPL* and the strong defensive effectiveness of *ES-Detector*.

II. RELATED WORK

1) Graph Prompt Learning: Graph prompt learning extends prompt-based adaptation to graph-structured data, enabling pretrained GNN encoders to adapt to downstream tasks without full parameter fine-tuning. ProG [1] attaches learnable prompt subgraphs to input graphs, while GraphPrompt [2] incorporates prompt tokens into latent representations to modulate graph readout. Subsequent studies improve prompt-based transfer in more challenging settings. IGAP [13] uses spectral-space prompting to reduce structural discrepancies in inductive scenarios. MultiGPrompt [14] adopts multi-task prompting to overcome the limitations of single-task pretraining, especially in few-shot learning. HgPrompt [15] extends prompt learning to heterogeneous graphs through a unified dual-template and dual-prompt design. Overall, these methods show that prompt mechanisms can improve the adaptability and transferability of pretrained GNN encoders across diverse graph structures and learning tasks.

2) Graph Backdoor Attacks: Backdoor attacks on GNNs have received increasing attention in recent years [6], [7], [8], [9], [10], [11], [16], [17], [18]. In such attacks, a backdoored GNN is manipulated to predict an attacker-specified label whenever a test graph contains a predefined trigger. Early studies generate trigger subgraphs using the Erdős-Rényi model [6], while later methods introduce adaptive trigger generators to improve attack effectiveness [10]. Other studies design label-preserving

triggers to improve stealthiness [7], or develop imperceptible graph backdoor attacks that evade existing defenses [9]. For graph contrastive learning, GCBA [19] poisons the victim GNN encoder by associating a trigger with the embedding of a target class. However, most existing attacks rely on prior knowledge of downstream tasks or target-class semantics. This assumption does not hold in GPL, where downstream tasks are unknown during pretraining and substantial distribution shifts are common. As a result, existing graph backdoor attacks have limited applicability in cross-context GPL settings.

III. PRELIMINARIES

A. Graphs and GNN Encoders

A graph is denoted as $G = (V, E)$, where $V = \{v_1, \dots, v_N\}$ is the set of N nodes and $E \subseteq V \times V$ is the set of edges. Each node v_i is associated with a d -dimensional feature vector $x_i \in \mathbb{R}^d$, and all node features are collected into a feature matrix $X \in \mathbb{R}^{N \times d}$. The graph structure is represented by an adjacency matrix $A \in \{0, 1\}^{N \times N}$, where $A_{i,j} = 1$ if $(v_i, v_j) \in E$. Graph neural networks (GNNs) learn node and graph representations through iterative neighborhood aggregation. At the k -th layer, the representation of node v_i is updated as $h_{v_i}^k = \text{AGGREGATE}(h_{v_i}^{k-1}, \{v_j \in \mathcal{N}(v_i) \cup v_i\}, \theta^{(k)})$, where $\mathcal{N}(v_i)$ denotes the set of neighbors of v_i , and $\theta^{(k)}$ represents the parameters of the k -th layer. The final node embedding is denoted as $h_{v_i} = E_\theta(G, v_i)$. A graph-level embedding $E_\theta(G)$ is obtained by applying a readout function over all node embeddings.

B. Graph Prompt Learning

We focus on two representative GPL frameworks, ProG [1] and GraphPrompt [2], both of which adapt pretrained GNN encoders to downstream tasks by introducing learnable prompts without modifying encoder parameters. While sharing this objective, they differ in how prompts are incorporated into the model [3]. ProG injects structured prompt subgraphs into the input graph, whereas GraphPrompt embeds prompt tokens into latent representations for graph pooling. ProG introduces a prompt graph $G_{pro} = (P, S)$, where $P = \{v_1^p, \dots, v_{|P|}^p\}$ denotes a set of prompt nodes with learnable token vectors $p_i \in \mathbb{R}^{1 \times d}$, and S defines the prompt graph structure. The prompt graph G_{pro} is integrated into the input graph G , and both the prompt parameters and the answering function ϕ are optimized via few-shot learning. GraphPrompt enhances the readout operation by introducing learnable prompt vectors into graph pooling. Given node embeddings $\{h_{v_i}\}_{v_i \in V}$, a task-specific prompt vector p_j modulates the readout via element-wise reweighting: $\text{Readout}(\{p_j \odot h_{v_i} \mid v_i \in V\})$, where $\odot$ denotes element-wise multiplication.

C. Cross-Context Few-Shot Learning

Cross-context few-shot learning aims to enable models to rapidly adapt to new tasks across diverse application contexts with limited labeled data. Due to data heterogeneity, substantial discrepancies often arise between pretraining and downstream stages in both semantic space and data distribution. Following

prior studies [1], [2], [3], [4], [5], we categorize cross-context scenarios into five representative types:

- *Cross-task*: Pretraining and downstream tasks differ in graph learning objectives, such as transferring from graph-level classification to node classification.
- *Cross-domain*: Datasets originate from different application domains but share the same task type, such as transferring from commercial product networks to academic citation networks.
- *Cross-dataset*: Pretraining and downstream data come from the same domain but different datasets, such as transferring from Cora to CiteSeer.
- *Cross-class*: Pretraining and downstream data share the same source but differ in label distributions.
- *Cross-distribution*: Both pretraining and downstream datasets share the same data origin, feature space, and label space, but exhibit substantial distribution shifts.

D. Threat Model

1) *Attacker's Goals*: The attacker aims to manipulate downstream models that rely on a compromised GNN encoder, such that inputs containing backdoor triggers are misclassified into an attacker-specified target class, while the model maintains normal behavior on clean inputs. In cross-context settings, the attacker implants the backdoor during the pretraining stage, enabling the encoder to learn a persistent association between backdoored inputs and adversarial output embeddings. When the backdoored encoder is adapted to downstream applications through GPL methods, the backdoor effect is transferred to downstream tasks.

To remain effective in cross-context GPL scenarios, an attack must be context-agnostic and prompt-agnostic, such that the implanted backdoor consistently activates across diverse downstream datasets, learning objectives, and prompt designs. At the same time, it must be stealthy, preserving performance on benign inputs at a level comparable to clean models and remaining effective even in the presence of downstream defense mechanisms.

2) *Attacker's Capabilities and Knowledge*: We consider a realistic backdoor threat in cross-context GPL scenarios, where a malicious service provider compromises pretrained GNN models distributed through online repositories. For example, a malicious third-party provider may release a pretrained GNN encoder to public model hubs or commercial graph learning platforms. Under this setting, the attacker has full control over the unlabeled pretraining dataset, the GNN architecture, and the pretraining paradigm, and can influence the pretraining process to embed persistent backdoor behaviors into the GNN encoder. However, the attacker has no access to labeled data or to the GPL procedures of downstream users, nor any knowledge of downstream datasets or prompt designs. Importantly, the attacker cannot modify existing nodes and can only inject a small number of new trigger nodes connected to selected anchor nodes. The attributes of these injected nodes are specified at creation time and are therefore controllable, while all existing nodes remain unchanged. Such capabilities are realistic in graph systems, where attackers can fabricate fake accounts or spurious entities and establish initial connections to existing nodes.

E. Defender's Goals and Capabilities

The defender aims to mitigate the impact of backdoored pre-trained GNN encoders in graph prompt learning by preventing downstream predictions from being influenced by embedded backdoor signals while preserving normal behavior on clean inputs. An effective defense should therefore suppress backdoor effects without degrading accuracy on backdoor-free data. Besides, the defense operates without prior knowledge of backdoor patterns, ensuring robustness against diverse attacks. We assume the defender has access to a small set of clean samples from multiple downstream classes, which serve as reference data for identifying anomalous behaviors induced by backdoor triggers. Notably, the defender can access and use the pretrained GNN encoder during downstream training and inference but is not allowed to modify or fine-tune its parameters, which remain frozen.

IV. OUR TROJANGPL ATTACK

A. Backdoor Vulnerability Analysis of GPL

We provide a theoretical analysis of the vulnerability of GPL to backdoor attacks. Following [1], we consider the prompt-as-graph setting, where a prompt graph G_{pro} is attached to an input graph G . Let D_s and D_t denote the pretraining and downstream graph distributions, respectively. A downstream prompted graph is constructed as $G_t \otimes G_{pro}$, and its embedding is given by $\hat{h}(G_t \otimes G_{pro}) = \hat{E}_{\theta_b}(G_t \otimes G_{pro})$.

Theorem 1. Upper Bound for the Backdoor Learning Loss at the Pretraining Stage. Let $\hat{E}_{\theta_b}$ be a $L_{\hat{E}}$ -Lipschitz continuous GNN encoder with input graphs containing k_G nodes. Suppose the node feature matrix X of a graph $G = (V, E)$ has a bounded Frobenius norm, i.e., $\|X\|_{fro} \leq \mu$. Upon freezing the GNN encoder $\hat{E}_{\theta_b}$, the backdoor learning loss $\mathcal{L}_{bkd}^s$ in the pretraining stage is upper-bounded by the main learning loss $\mathcal{L}_{clr}^s$ in the same stage, as follows:

$$\mathcal{L}_{bkd}^s \leq \mathcal{L}_{clr}^s + 2\sqrt{d(\Delta_G^+, G_s \oplus \Delta_G)} + C_0 \quad (1)$$

where the function $d(\Delta_G^+, G_s \oplus \Delta_G)$ is given by:

$$\begin{aligned} d(\Delta_G^+, G_s \oplus \Delta_G) &= 2\sqrt{k_G}(k_G - 1)L_{\hat{E}}\mu \\ &\quad - \sum_{i=1}^n \sum_{j=1}^m \frac{s(\hat{h}(\Delta_{G,i}^+), \hat{h}(G_{s,j} \oplus \Delta_G))}{nm}. \end{aligned} \quad (2)$$

Here, Δ_G^+ represents augmented trigger graphs obtained by randomly adding or removing edges from Δ_G . The function $s(\cdot)$ denotes a similarity measure between graph embeddings in the Reproducing Kernel Hilbert Space (RKHS). G_s denotes graphs sampled from the pretraining data distributions D_s . The constant C_0 depends on the complexity of the GNN encoder and the optimal learning loss of an ideal encoder.

Theorem 2. Upper Bound for the Main Learning Loss at the Downstream Stage. Under the same assumptions as Theorem 1, the main learning loss $\mathcal{L}_{clr}^t$ in the downstream application can be upper-bounded by the main learning loss $\mathcal{L}_{clr}^s$

in the pretraining stage as follows:

$$\mathcal{L}_{clr}^t \leq \mathcal{L}_{clr}^s + 2\sqrt{d(G_s, G_t \otimes G_{pro})} + C_1 \quad (3)$$

where the function $d(G_s, G_t \otimes G_{pro})$ is given by:

$$\begin{aligned} d(G_s, G_t \otimes G_{pro}) &= 2\sqrt{k_G}(k_G - 1)L_{\hat{E}}\mu \\ &\quad - \sum_{i=1}^n \sum_{j=1}^m \frac{s(\hat{h}(G_{s,i}), \hat{h}(G_{t,j} \otimes G_{pro}))}{nm}. \end{aligned} \quad (4)$$

Here, G_t denote graphs sampled from the downstream dataset D_t . The constant C_1 depends on the complexity of the GNN encoder and the optimal learning loss of an ideal encoder.

Theorem 3. Upper Bound for the Backdoor Learning Loss at the Downstream Stage. Under the same assumptions as Theorem 1, the backdoor learning loss $\mathcal{L}_{bkd}^t$ in the downstream application is upper-bounded as follows:

$$\begin{aligned} \mathcal{L}_{bkd}^t &\leq \mathcal{L}_{clr}^s + 2\sqrt{d(\Delta_G^+, G_s \oplus \Delta_G)} \\ &\quad + 2\sqrt{d(G_s \oplus \Delta_G, (G_t \oplus \Delta_G) \otimes G_{pro})} + C_2 \end{aligned} \quad (5)$$

where $d(G_s \oplus \Delta_G, (G_t \oplus \Delta_G) \otimes G_{pro})$ is given by:

$$\begin{aligned} d(G_s \oplus \Delta_G, (G_t \oplus \Delta_G) \otimes G_{pro}) &= 2\sqrt{k_G}(k_G - 1)L_{\hat{E}}\mu \\ &\quad - \sum_{i=1}^n \sum_{j=1}^m \frac{s(\hat{h}(G_{s,i} \oplus \Delta_G), \hat{h}((G_{t,j} \oplus \Delta_G) \otimes G_{pro}))}{nm} \end{aligned} \quad (6)$$

Here, C_2 is the constant for the GNN encoder's complexity and the optimal learning loss of an ideal encoder

Observation 1. Transferability of Graph Prompt Learning in Boosting Backdoor Attacks.

Proposition 1. Given a backdoored GNN encoder $\hat{E}_{\theta_b}$, there exists a prompt graph $\sum_{i=1}^n \sum_{j=1}^m \frac{s(\hat{h}(G_{s,i}), \hat{h}(G_{t,j} \otimes G_{pro}))}{nm}$, which measures the similarity between graphs from the pre-training dataset and the prompted graphs from the downstream dataset.

Proposition 1 shows that, for a backdoored GNN encoder, an appropriately designed prompt graph can align the embedding distributions of pretraining graphs and prompted downstream graphs. This alignment reduces the discrepancy between the main learning losses in both stages, denoted as $\mathcal{L}_{clr}^s$ and $\mathcal{L}_{clr}^t$. As a result, the upper bound of the downstream main task loss is tightened, demonstrating why graph prompt learning effectively transfers pretrained knowledge across contexts. Importantly, the same alignment mechanism also applies to the backdoor objective, bridging the gap between the backdoor learning losses $\mathcal{L}_{bkd}^s$ and $\mathcal{L}_{bkd}^t$. This dual effect of prompt alignment on both main and backdoor objectives is summarized as Observation 1.

Observation 2. Trigger Graph Tuning Enhances the Transferability of Backdoor Attacks.

Proposition 2. Given a backdoored GNN encoder $\hat{E}_{\theta_b}$, there exists a trigger graph Δ_G that maximizes $\frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m s(\hat{h}(\Delta_{G,i}^+), \hat{h}(G_{s,j} \oplus \Delta_G))$, which measures the similarity between augmented variants of the trigger graph and backdoored graphs during pretraining.

Proposition 2 demonstrates that the trigger graph can be optimized to align its augmented embeddings with those of backdoored graphs observed during pretraining. This alignment reduces the discrepancy between the main learning loss and the backdoor learning loss in the pretraining stage, denoted as $\mathcal{L}_{\text{clr}}^s$ and $\mathcal{L}_{\text{bkd}}^s$, respectively. Consequently, the upper bound of the backdoor learning loss in downstream applications is further reduced. This optimization allows the use of an optimized trigger graph with a GNN encoder that maintains strong performance on the main task, enabling both classification losses on clean and backdoored graphs to be minimized simultaneously. As a result, the attacker can amplify the backdoor poisoning effects while preserving the utility of the pretrained GNN encoder on clean data. We formalize this insight as Observation 2.

Summary. GPL bridges the gap between pretraining and downstream tasks, facilitating the transfer of knowledge for the main learning objective. However, this mechanism also enables the propagation of backdoor behaviors embedded in pretrained encoders. Moreover, by explicitly tuning the trigger graph to align the loss landscapes of both the main and backdoor objectives, the transferability and persistence of backdoor effects can be amplified across downstream contexts. Building on these theoretical insights, we propose *TrojanGPL*, a method that enhances the transferability of backdoor poisoning from pretrained GNN encoders to downstream applications. This is achieved through the joint use of prompt learning and trigger graph optimization, which is guided by loss alignment and embedding alignment.

B. Attack Design of TrojanGPL

Building on the above analysis, we propose *TrojanGPL*, a cross-context backdoor attack against GPL. *TrojanGPL* constructs an embedding collision in the pretrained GNN encoder between a graph injected with an attacker-designed trigger and the trigger graph itself, while keeping backdoored graphs distinguishable from clean ones in the embedding space. Under this design, the encoder is forced to output an attacker-defined target embedding for any input graph containing the trigger. This target embedding is intentionally separated from typical clean representations, which causes downstream GPL models to produce attacker-intended misclassifications once the trigger is activated.

1) *Trigger Graph Design in TrojanGPL*: The trigger graph, denoted as $\Delta_G = (V_\Delta, E_\Delta)$, serves as the backdoor signal. Specifically, $V_\Delta = \{v_\Delta^1, \dots, v_\Delta^C\}$ is a set of C trigger nodes, where each node v_Δ^i is assigned a feature vector $x_\Delta^i \in \mathbb{R}^{1 \times d}$ that matches the feature dimensionality of the input graph. The edge set $E_\Delta = \{(v_\Delta^i, v_\Delta^j) \mid v_\Delta^i, v_\Delta^j \in V_\Delta\}$ defines the connectivity structure of the trigger graph. To inject Δ_G into an input graph G and obtain a backdoored graph, the attacker selects a node in G as an anchor node v_{att} and connects v_{att} to a designated trigger node v_Δ^i . For stealthiness and efficiency, we constrain Δ_G to be a fully connected three-node subgraph, which is substantially smaller than the graphs used in both pretraining and downstream tasks.

2) *Target Embedding Design in TrojanGPL*: Since the attacker has no prior knowledge of the downstream data

distribution or label space, we define the target embedding as the encoder output on the trigger graph itself. This choice makes the attack independent of any specific downstream label space. After the trigger is introduced into a downstream GPL model, the attacker can infer the label associated with this target embedding under the downstream classifier, thereby enabling the backdoor to be executed in a task-agnostic manner.

3) *Overall Learning Objectives of TrojanGPL*: We formulate *TrojanGPL* as a joint optimization problem that balances clean representation learning and backdoor injection:

$$\theta_b^*, \Delta_G^* = \arg \min_{\theta_b, \Delta_G} \mathcal{L}_{\text{cIn}} + \mathcal{L}_{\text{bkd}} + \alpha \mathcal{L}_{\text{sim}}^c + \beta \mathcal{L}_{\text{sim}}^a. \quad (7)$$

Here, α and β control the relative importance of the regularization terms. The four terms in (7) serve complementary purposes, which we describe below.

a) *Clean Representation Learning Loss*: To ensure that the backdoored GNN encoder maintains strong representation quality on clean graphs, *TrojanGPL* incorporates a standard contrastive learning objective:

$$\mathcal{L}_{\text{cIn}} = \sum_{G_i \in D} \log \frac{\exp(\text{sim}(H_{G_i}, H_{G_i^+})/\tau)}{\exp(\text{sim}(H_{G_i}, H_{G_i^+})/\tau) + \sum_{k \neq i} \exp(\text{sim}(H_{G_i}, H_{G_k^-})/\tau)} \quad (8)$$

where G_i is a clean graph sampled from the pretraining dataset D , G_i^+ is an augmented view of G_i obtained by randomly adding or removing edges, and G_k^- denotes a different graph in D . τ is the temperature parameter, and $\text{sim}(\cdot, \cdot)$ denotes a similarity function. $H_{G_i} = \hat{E}_{\theta_b}(G_i)$ is the embedding produced by the backdoored encoder. This objective encourages invariant representations for augmented views of the same graph while maintaining discriminative embeddings across different graphs, thereby preserving main-task utility.

b) *Backdoor Injection Loss*: The objective of *TrojanGPL* is to embed a backdoor that maps trigger-injected graphs to an attacker-defined target embedding. Since the attacker has no prior knowledge of downstream label spaces, *TrojanGPL* uses the embedding of the trigger graph itself as the target embedding. The backdoor injection loss is defined as:

$$\mathcal{L}_{\text{bkd}} = -\frac{1}{|D|} \sum_{G_i \in D} \text{sim}(\hat{E}_{\theta_b}(G_i \oplus \Delta_G), \hat{E}_{\theta_b}(\Delta_G)) + \lambda \frac{1}{|D|} \sum_{G_i \in D} \text{sim}(\hat{E}_{\theta_b}(G_i \oplus \Delta_G), \hat{E}_{\theta_b}(G_i)) \quad (9)$$

where $\oplus$ attaches the trigger graph Δ_G to a clean graph G_i via the anchor node v_{att} , and λ controls the relative importance of the two terms. This loss aligns trigger-injected graphs with the embedding of trigger graphs, while discouraging similarity to the corresponding clean embeddings. As a result, graphs containing the trigger are consistently mapped to the target embedding and misclassified in downstream tasks.

c) *Clean Utility Preservation Loss*: To further preserve normal behavior on benign inputs, *TrojanGPL* introduces an embedding

alignment loss with a clean reference encoder:

$$\mathcal{L}_{\text{sim}}^c = -\frac{1}{|D|} \sum_{G_i \in D} \text{sim}(\hat{E}_{\theta_b}(G_i), E_{\theta_c}(G_i)) \quad (10)$$

where E_{θ_c} is a clean GNN encoder pretrained on clean data. This loss constrains the backdoored encoder to produce embeddings consistent with those of a clean model on clean graphs, thereby mitigating utility degradation.

d) *Stealth Enhancement Loss*: To improve stealth against feature-based anomaly detectors, *TrojanGPL* further incorporates a node feature affinity loss:

$$\mathcal{L}_{\text{sim}}^a = -\frac{1}{|D|} \sum_{G_i \in D} \sum_{v_{\Delta}^j \in V_{\Delta}} \text{sim}(\text{feat}(v_{\Delta}^j), \text{feat}(v_{\text{att}})), \quad (11)$$

where $\text{feat}(\cdot)$ denotes the node feature vector and v_{att} is the anchor node. This loss encourages trigger nodes to exhibit feature distributions similar to surrounding nodes, making the injected trigger graph less distinguishable from the original graph structure.

e) *Optimization Strategy*: In practice, *TrojanGPL* solves the joint optimization problem in (7) using an alternating optimization strategy. Specifically, the attacker iteratively updates the encoder parameters θ_b and the trigger graph Δ_G while keeping the other fixed, until convergence.

C. Evaluation of TrojanGPL

1) Experimental Settings: Datasets. We evaluate *TrojanGPL* on five benchmark datasets: CiteSeer and Cora [20], Amazon-Computers and Amazon-Photo [21], and ENZYMEs [22]. CiteSeer, Cora, Amazon-Computers, and Amazon-Photo are used for node classification, while ENZYMEs is used for graph classification. Following [1], we additionally construct graph classification tasks from node classification datasets.

GPL Methods. We evaluate two categories of GPL methods [3]: *Prompt as Graphs*, represented by ProG [1], and *Prompt as Tokens*, represented by GraphPrompt [2]. We consider two representative GNN backbones, Graph Attention Network (GAT) [23] and Graph Transformer (GT) [24].

Baseline Attacks. The closest baseline under our threat model is GCBA [19], which poisons a GNN encoder during graph contrastive learning. GCBA is not designed for GPL and assumes access to downstream information to construct backdoors, which limits its applicability in cross-context settings. To adapt GCBA to our threat model, we consider two variants, GCBA_R and GCBA_M. Both cluster embeddings of clean pretraining graphs produced by a backdoor-free encoder. GCBA_R randomly selects a cluster centroid as the target embedding, while GCBA_M chooses the centroid that is maximally separated from the others.

Evaluation Metrics. We report Attack Success Rate (ASR), Main Task Accuracy (ACC), and Accuracy Drop (AD), defined as the ACC degradation from a backdoor-free model to a backdoored model. An effective attack should achieve high ASR with high ACC and low AD.

2) Attack Performance Without Defense: We evaluate the feasibility of backdoor attacks across five cross-context scenarios using three mainstream GPL methods. Tables I, II, III, IV, and V summarize the ACC and ASR results for all attack methods in different cross-context scenarios. The highest ASR and ACC values, as well as the lowest AD values among all evaluated attacks are highlighted in bold.

Overall, *TrojanGPL* achieves the highest ASR while maintaining utility close to that of backdoor-free models. Across the reported settings, it consistently attains ASR above 0.87 and limits the ACC drop to within 0.06, whereas baseline attacks often fail to preserve both ASR and ACC.

Across five cross-context scenarios, *TrojanGPL* achieves consistently high transferability while preserving clean utility. In cross-distribution and cross-class settings, it reaches ASR above 0.90 with negligible accuracy degradation. In cross-dataset and cross-domain settings, it maintains strong transfer with ASR above 0.80. In cross-task settings, it still achieves ASR above 0.85, indicating robust generalization across different learning objectives. Overall, *TrojanGPL* exhibits stable backdoor transfer across diverse GPL deployments.

Overall, these findings confirm that *TrojanGPL* effectively generalizes backdoor poisoning effects across diverse cross-context scenarios, outperforming two baseline methods while preserving the utility of GNN encoders for downstream tasks.

3) Attack Performance Against Defenses: We further explore defenses by adapting existing techniques, as, to our knowledge, no dedicated defense has been proposed for cross-context GPL. Real-world graphs often exhibit homophily, where connected nodes tend to have similar features. Injecting trigger subgraphs can violate this property. Accordingly, we adapt *PruneG* [25], which prunes edges between dissimilar nodes based on a similarity threshold. Under our threat model, the downstream user applies pruning and retains the major connected component after pruning.

Table VI summarizes results under *PruneG* for node and graph classification across five cross-context scenarios and two GPL methods. In cross-dataset settings, Cora is used for pretraining. Photo is used for cross-domain settings, and an induced graph dataset is used for cross-task settings.

Under *PruneG*, baseline attacks exhibit consistently poor performance. In contrast, *TrojanGPL* remains effective, achieving high ASR across settings. For example, in cross-domain scenarios, *TrojanGPL* attains ASR above 0.90 in all the tested cases, while baselines remain below 0.50. This robustness is attributable to the optimization that aligns trigger node features with those of anchor nodes, which reduces pruning-induced disruption of the trigger.

4) Ablation Study: We conduct ablation studies to quantify the contribution of each component in *TrojanGPL*. We evaluate three variants: (1) w/o TO, which removes trigger graph optimization; (2) w/o EA, which removes the clean embedding alignment loss; and (3) w/o NFA, which removes the node feature affinity constraint. Table VII reports their performance against two GPL methods under the *PruneG* defense across 5 cross-context scenarios. Removing trigger optimization leads to the largest ASR drop, demonstrating that explicit trigger and

TABLE I
ACC, ASR, AND AD OF ATTACK METHODS IN CROSS-DISTRIBUTION SCENARIOS

GPL	Model	Attack	Node Classification								Graph Classification			
			CiteSeer		Cora		Computers		Photo		CiteSeer-Graph		Photo-Graph	
			ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR
ProG	GAT	GCBA_R	0.24 (+0.60)	0.51	0.33 (+0.32)	0.76	0.43 (+0.29)	0.18	0.61 (+0.18)	0.15	0.17 (+0.54)	0.00	0.66 (+0.23)	0.24
		GCBA_M	0.24 (+0.60)	0.62	0.18 (+0.47)	0.00	0.40 (+0.32)	0.44	0.65 (+0.14)	0.18	0.17 (+0.54)	0.00	0.69 (+0.20)	0.15
		TrojanGPL	0.83 (+0.01)	0.90	0.64 (+0.01)	1.00	0.70 (+0.02)	1.00	0.79 (-0.00)	0.94	0.76 (-0.05)	1.00	0.89 (-0.00)	0.91
	GT	GCBA_R	0.35 (+0.47)	0.96	0.35 (+0.38)	0.69	0.47 (+0.27)	1.00	0.57 (+0.22)	0.85	0.37 (+0.42)	0.32	0.58 (+0.33)	0.89
		GCBA_M	0.35 (+0.47)	0.96	0.45 (+0.28)	0.23	0.41 (+0.33)	0.75	0.60 (+0.19)	0.30	0.33 (+0.46)	0.21	0.57 (+0.34)	0.56
		TrojanGPL	0.82 (-0.00)	1.00	0.73 (-0.00)	0.99	0.72 (+0.02)	1.00	0.79 (-0.00)	1.00	0.80 (-0.01)	1.00	0.92 (-0.01)	0.99
Graph Prompt	GAT	GCBA_R	0.69(+0.07)	0.05	0.47 (+0.11)	0.14	0.49 (+0.11)	0.14	0.59 (+0.07)	0.00	0.72 (-0.00)	0.04	0.58 (+0.20)	0.00
		GCBA_M	0.78 (-0.02)	0.01	0.43 (+0.15)	0.07	0.43 (+0.17)	0.13	0.56 (+0.10)	0.15	0.67 (+0.05)	0.13	0.51 (+0.27)	0.28
		TrojanGPL	0.80 (-0.04)	1.00	0.67 (-0.09)	1.00	0.62 (-0.02)	1.00	0.68 (-0.02)	0.99	0.72 (-0.00)	0.99	0.74 (+0.04)	1.00
	GT	GCBA_R	0.69 (+0.09)	0.16	0.56 (+0.15)	0.19	0.50 (+0.15)	0.15	0.62 (+0.13)	0.13	0.66 (+0.09)	0.10	0.55 (+0.29)	0.25
		GCBA_M	0.64 (+0.14)	0.04	0.53 (+0.18)	0.25	0.47 (+0.18)	0.10	0.59 (+0.16)	0.12	0.74 (+0.01)	0.11	0.57 (+0.27)	0.14
		TrojanGPL	0.79 (-0.01)	1.00	0.70 (+0.01)	0.99	0.67 (-0.02)	1.00	0.74 (+0.01)	1.00	0.77 (-0.02)	1.00	0.84 (-0.00)	1.00

TABLE II
ACC, ASR, AND AD OF ATTACK METHODS IN CROSS-CLASS SCENARIOS

GPL	Model	Attack	Node Classification								Graph Classification			
			CiteSeer		Cora		Computers		Photo		CiteSeer-Graph		Photo-Graph	
			ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR
ProG	GAT	GCBA_R	0.25 (+0.68)	0.00	0.47 (+0.31)	1.00	0.55 (+0.28)	1.00	0.73 (+0.06)	0.21	0.26 (+0.62)	0.00	0.80 (+0.12)	0.84
		GCBA_M	0.25 (+0.68)	0.00	0.49 (+0.29)	1.00	0.56 (+0.27)	0.95	0.68 (+0.11)	0.36	0.46 (+0.41)	1.00	0.79 (+0.13)	1.00
		TrojanGPL	0.93 (-0.00)	1.00	0.78 (-0.00)	1.00	0.83 (-0.00)	1.00	0.79 (-0.00)	1.00	0.86 (+0.01)	1.00	0.93 (-0.01)	1.00
	GT	GCBA_R	0.49 (+0.44)	0.29	0.61 (+0.27)	0.99	0.60 (+0.22)	0.72	0.62 (+0.13)	1.00	0.49 (+0.39)	0.46	0.76 (+0.10)	0.01
		GCBA_M	0.49 (+0.44)	0.34	0.52 (+0.36)	1.00	0.70 (+0.12)	1.00	0.69 (+0.06)	0.71	0.66 (+0.22)	0.26	0.80 (+0.06)	0.80
		TrojanGPL	0.93 (-0.00)	1.00	0.88 (-0.00)	1.00	0.82 (-0.00)	1.00	0.76 (-0.01)	1.00	0.88 (-0.00)	1.00	0.88 (-0.02)	1.00
Graph Prompt	GAT	GCBA_R	0.92 (-0.04)	0.05	0.86 (-0.01)	0.07	0.69 (+0.06)	0.10	0.74 (-0.04)	0.01	0.86 (-0.03)	0.06	0.61 (+0.17)	0.28
		GCBA_M	0.89 (-0.01)	0.02	0.81 (+0.04)	0.03	0.70 (+0.05)	0.06	0.60 (+0.10)	0.00	0.84 (-0.00)	0.06	0.66 (+0.11)	0.27
		TrojanGPL	0.92 (-0.04)	1.00	0.86 (-0.02)	1.00	0.77 (-0.02)	1.00	0.72 (-0.02)	0.94	0.83 (+0.01)	1.00	0.79 (-0.01)	1.00
	GT	GCBA_R	0.87 (+0.06)	0.03	0.76 (+0.11)	0.03	0.72 (+0.08)	0.08	0.66 (+0.12)	0.03	0.88 (-0.01)	0.05	0.59 (+0.21)	0.24
		GCBA_M	0.88 (+0.05)	0.05	0.83 (+0.04)	0.00	0.70 (+0.10)	0.05	0.66 (+0.12)	0.15	0.87 (-0.00)	0.04	0.66 (+0.14)	0.15
		TrojanGPL	0.93 (-0.00)	1.00	0.88 (-0.01)	1.00	0.80 (-0.00)	1.00	0.79 (-0.01)	1.00	0.89 (-0.02)	1.00	0.87 (-0.07)	1.00

TABLE III
ACC, ASR, AND AD IN CROSS-DATASET SCENARIOS. CORA IS USED AS THE PRETRAINING DATASET FOR CITESEER AND CITESEER-GRAPH, AND COMPUTERS IS USED FOR PHOTO AND PHOTO-GRAPH

GPL	Model	Attack	Node Classification				Graph Classification			
			CiteSeer		Photo		CiteSeer-Graph		Photo-Graph	
			ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR
ProG	GAT	GCBA_R	0.24 (+0.57)	0.48	0.44 (+0.34)	0.59	0.17 (+0.61)	0.00	0.59 (+0.30)	0.28
		GCBA_M	0.19 (+0.62)	0.00	0.46 (+0.32)	0.28	0.17 (+0.61)	0.06	0.44 (+0.45)	0.68
		TrojanGPL	0.83 (-0.02)	1.00	0.78 (-0.00)	1.00	0.79 (-0.01)	1.00	0.89 (-0.00)	0.97
	GT	GCBA_R	0.48 (+0.34)	0.80	0.55 (+0.26)	0.95	0.44 (+0.35)	0.42	0.67 (+0.27)	0.99
		GCBA_M	0.48 (+0.34)	0.90	0.52 (+0.29)	0.16	0.44 (+0.35)	0.75	0.58 (+0.36)	0.62
		TrojanGPL	0.82 (-0.00)	1.00	0.81 (-0.00)	1.00	0.79 (-0.00)	1.00	0.93 (+0.01)	1.00
Graph Prompt	GAT	GCBA_R	0.72 (-0.00)	0.09	0.60 (+0.12)	0.12	0.67 (+0.06)	0.10	0.71 (+0.10)	0.10
		GCBA_M	0.59 (+0.13)	0.12	0.64 (+0.08)	0.03	0.64 (+0.09)	0.03	0.65 (+0.16)	0.16
		TrojanGPL	0.67 (+0.05)	0.97	0.70 (+0.02)	1.00	0.67 (+0.06)	0.93	0.82 (-0.01)	1.00
	GT	GCBA_R	0.66 (+0.13)	0.06	0.59 (+0.13)	0.13	0.59 (+0.18)	0.04	0.68 (+0.16)	0.09
		GCBA_M	0.55 (+0.24)	0.01	0.58 (+0.14)	0.12	0.66 (+0.11)	0.09	0.66 (+0.18)	0.32
		TrojanGPL	0.80 (-0.01)	1.00	0.71 (+0.01)	0.99	0.77 (-0.00)	1.00	0.85 (+0.01)	0.93

target embedding optimization is essential for forming stable embedding collisions across prompts and contexts. Removing embedding alignment degrades both ASR and utility by allowing backdoor optimization to disrupt clean representation geometry. Removing node feature affinity mainly reduces robustness under pruning-based defenses, as trigger nodes become more distinguishable from their neighborhood.

5) *Impact of Prompt Tokens*: Fig. 1 shows the impact of prompt token count on *TrojanGPL* against ProG across five cross-context scenarios. *TrojanGPL* remains stable as the number of prompt tokens varies, indicating that the attack is not sensitive to prompt capacity.

6) *Impact of Trigger Nodes*: Fig. 2 shows the effect of trigger node count on *TrojanGPL* against ProG and GraphPrompt across

TABLE IV
ACC, ASR, AND AD IN CROSS-DOMAIN SCENARIOS. PHOTO IS USED AS THE PRETRAINING DATASET

GPL	Model	Attack	Node Classification				Graph Classification			
			CiteSeer		Cora		CiteSeer-Graph		Cora-Graph	
			ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR	ACC (AD)	ASR
ProG	GAT	GCBA_R	0.24 (+0.50)	1.00	0.18 (+0.40)	1.00	0.31 (+0.41)	0.80	0.14 (+0.74)	0.00
		GCBA_M	0.20 (+0.54)	0.00	0.32 (+0.26)	1.00	0.20 (+0.52)	0.88	0.14 (+0.74)	0.00
		TrojanGPL	0.76 (-0.02)	1.00	0.73 (-0.15)	1.00	0.72 (-0.00)	1.00	0.85 (+0.03)	1.00
	GT	GCBA_R	0.40 (+0.43)	1.00	0.34 (+0.39)	0.00	0.33 (+0.47)	1.00	0.14 (+0.76)	0.00
		GCBA_M	0.42 (+0.41)	1.00	0.33 (+0.40)	1.00	0.44 (+0.36)	1.00	0.40 (+0.50)	1.00
		TrojanGPL	0.83 (-0.00)	1.00	0.73 (-0.00)	1.00	0.80 (-0.00)	1.00	0.90 (-0.00)	1.00
Graph Prompt	GAT	GCBA_R	0.75 (-0.03)	0.85	0.68 (-0.00)	0.24	0.59 (+0.15)	0.00	0.78 (+0.05)	0.60
		GCBA_M	0.77 (-0.05)	0.71	0.58 (+0.10)	0.00	0.63 (+0.11)	0.03	0.64 (+0.19)	0.23
		TrojanGPL	0.76 (-0.04)	1.00	0.70 (-0.02)	1.00	0.74 (-0.00)	1.00	0.81 (+0.02)	1.00
	GT	GCBA_R	0.64 (+0.15)	0.19	0.57 (+0.14)	0.02	0.68 (+0.08)	0.00	0.74 (+0.10)	0.04
		GCBA_M	0.70 (+0.09)	0.00	0.53 (+0.18)	0.00	0.58 (+0.18)	0.00	0.76 (+0.08)	0.00
		TrojanGPL	0.79 (-0.00)	0.99	0.72 (-0.01)	0.99	0.77 (-0.01)	1.00	0.82 (+0.02)	1.00

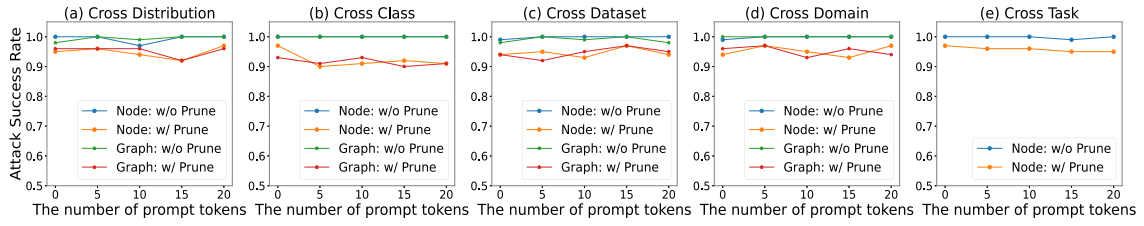


Fig. 1. Impact of the number of prompt tokens on attack performance of TrojanGPL against ProG based on CiteSeer in 5 cross-context scenarios. "Node" represents the node classification task, and "Graph" denotes the graph classification task.

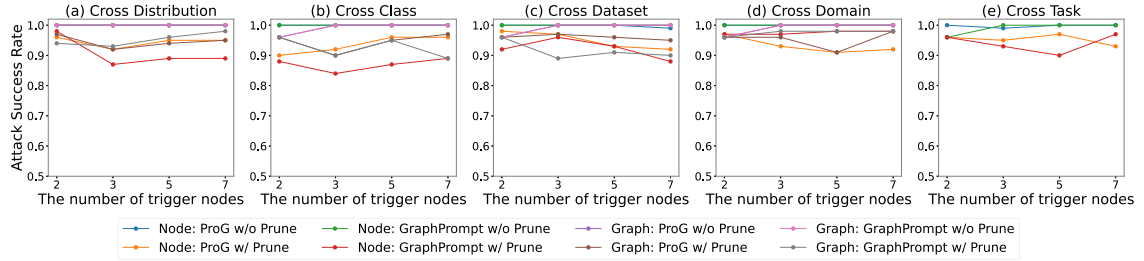


Fig. 2. Impact of the number of trigger nodes on performance of TrojanGPL against ProG and GraphPrompt based on CiteSeer in 5 cross-context scenarios. "Node" represents the node classification task, and "Graph" denotes the graph classification task.

five scenarios. *TrojanGPL* consistently achieves ASR above 0.80 across trigger sizes, indicating robustness and efficiency with small triggers.

V. THE ES-DETECTOR DEFENSE

A. Key Intuition

Our key intuition is that clean graphs assigned to the same class by the downstream model tend to exhibit consistent embedding shifts after important-node removal, whereas backdoored graphs show markedly different shifts even when they are predicted as the same class. Here, an important node refers to the node whose removal causes the largest change in the graph embedding. Fig. 3 illustrates the distribution of embedding shifts after important-node removal and compares their consistency between clean and backdoored graphs with the same predicted class.

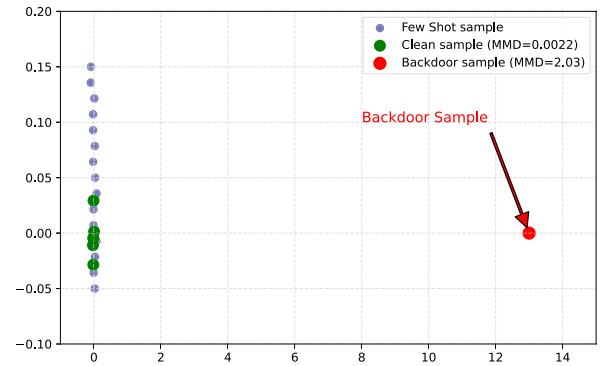


Fig. 3. Graph embedding shifts after removing the important node, measured by MMD distance, for clean and backdoored graphs classified into the same class.

As shown in Fig. 3, few-shot clean samples shown in blue serve as references for typical embedding variations. Clean

TABLE V
ACC, ASR, AND AD IN CROSS-TASK SCENARIOS. ENZYMES IS THE
PRETRAINING DATASET

GPL	Model	Attack	CiteSeer		Cora	
			ACC (AD)	ASR	ACC (AD)	ASR
ProG	GAT	GCBA_R	0.50 (+0.26)	0.23	0.24 (+0.42)	0.00
		GCBA_M	0.61 (+0.15)	0.10	0.24 (+0.42)	0.00
		TrojanGPL	0.75 (+0.01)	1.00	0.72 (-0.06)	1.00
	GT	GCBA_R	0.55 (+0.15)	0.85	0.44 (+0.20)	0.80
		GCBA_M	0.57 (+0.13)	0.77	0.44 (+0.20)	0.93
		TrojanGPL	0.82 (-0.12)	0.87	0.64 (-0.00)	1.00
Graph Prompt	GAT	GCBA_R	0.70 (+0.04)	0.03	0.26 (+0.38)	0.41
		GCBA_M	0.71 (+0.03)	0.00	0.34 (+0.30)	0.13
		TrojanGPL	0.75 (-0.01)	1.00	0.65 (-0.01)	0.97
	GT	GCBA_R	0.69 (+0.06)	0.05	0.56 (-0.03)	0.24
		GCBA_M	0.77 (-0.02)	0.04	0.50 (+0.03)	0.26
		TrojanGPL	0.75 (-0.00)	1.00	0.53 (-0.00)	1.00

graphs shown in green form a compact distribution after important-node removal, indicating consistent embedding shifts within the same predicted class. By contrast, the backdoored graph shown in red exhibits a substantially different shift, which deviates from both the few-shot references and the clean samples. The large MMD distance further confirms that backdoored graphs introduce anomalous embedding deviations that are separable from clean shift patterns. Based on this observation, ES-Detector detects backdoored graphs by exploiting the consistency of embedding shifts induced by important-node removal.

This distinction arises because clean graphs predicted as the same class usually share similar task-relevant structural and semantic patterns. Thus, removing the most important node mainly weakens comparable class-relevant cues, leading to stable and predictable embedding shifts. In contrast, backdoored graphs are partly dominated by trigger-induced shortcuts rather than genuine class semantics. Removing the important node may disrupt the interaction among the trigger, the anchor node, and the surrounding graph context, causing the embedding to shift in a direction inconsistent with clean reference graphs. This observation motivates ES-Detector to identify backdoored graphs by comparing the embedding-shift pattern of a test graph with those of clean reference graphs from the same predicted class.

B. Defense Design of ES-Detector

ES-Detector is a defense mechanism for detecting and mitigating backdoor triggers in graph prompt learning. It operates by analyzing the sensitivity of graph embeddings to node removal, based on the observation that backdoor triggers induce abnormal embedding shifts that differ from those of clean graphs. For a given input graph, ES-Detector identifies influential nodes, evaluates the resulting embedding shift against reference behavior derived from clean samples, and iteratively removes anomalous nodes when necessary. The detailed design of each component is described below.

1) *Selection of Important Nodes*: To identify the most important nodes within a given graph, the defender can perform a systematic sensitivity analysis by evaluating the impact of removing individual nodes while keeping the rest of the graph unchanged. For a graph $G = (V, E)$, let $E_\theta(G)$ denote its graph

embedding produced by the pretrained encoder. For each node $v_i \in V$, we construct a perturbed graph G_{-v_i} by removing v_i and all incident edges, and compute the corresponding embedding $E_\theta(G_{-v_i})$. We quantify the similarity between the original and perturbed embeddings using cosine similarity:

$$\text{Sim}(v) = \frac{E_\theta(G) \cdot E_\theta(G_{-v_i})}{\|E_\theta(G)\| \|E_\theta(G_{-v_i})\|}. \quad (12)$$

The node whose removal yields the lowest similarity is defined as the most important node $v^* = \arg \min_{v_i \in G} \text{Sim}(v_i)$. This node will be used to construct embedding-shift features for detection and to guide subsequent pruning.

2) *Selection of Pruning Threshold*: In the GPL system, the defender has access to a small set of clean samples D'_t associated with the downstream task. We use these samples to determine a threshold that separates clean graphs from potential backdoored graphs based on their embedding-shift behavior. For any graph G , we define the embedding shift induced by removing its important node as $\Delta E_\theta(G) = E_\theta(G) - E_\theta(G_{-v^*})$. We compute $\Delta E_\theta(G)$ for clean samples in D'_t and estimate the reference distribution of embedding shifts. The defender then measures distributional discrepancy using MMD over these embedding-shift features. This statistical measure quantifies the distributional differences in embedding variations caused by critical node removal. Based on this distribution, we select a threshold ϵ to distinguish clean samples from outliers that exhibit abnormal embedding-shift patterns.

3) *Anomalous Node Removal*: Given a test graph G_{test} , the defender first computes its important node $v^*(G_{\text{test}})$ and the corresponding embedding shift $\Delta E_\theta(G_{\text{test}})$. It then evaluates the MMD distance between the test embedding shift and the reference embedding shifts of clean support samples $d_{\text{test}} = \text{MMD}(\Delta E_\theta(G_{\text{test}}), \{\Delta E_\theta(G_i)\}_{i=1}^N)$, where $\{G_i\}_{i=1}^N$ are clean samples in D'_t .

We apply a threshold-based decision rule. If $d_{\text{test}} \leq \epsilon$, the input is treated as clean and is forwarded to the downstream model without modification. Otherwise, the input is flagged as potentially backdoored. In this case, the defender removes the current important node $v^*(G_{\text{test}})$ and its incident edges to obtain an updated graph, and repeats the same detection procedure on the updated graph. This iterative pruning continues until the resulting distance satisfies $d_{\text{test}} \leq \epsilon$. The processed graph is then fed into the downstream model for final prediction.

C. Theoretical Analysis of ES-Detector

We present a theoretical analysis that explains why ES-Detector can effectively detect and mitigate backdoor triggers in graph prompt learning. The analysis focuses on the representation-level response of graph embeddings to node-level perturbations.

Let $G = (V, E)$ denote an input graph, and let $E_\theta(G) \in \mathbb{R}^d$ be the graph-level embedding produced by a pretrained GNN encoder with parameters θ . For any node $v \in V$, we define the embedding perturbation induced by node removal as $\Delta z_\theta(G, v) = E_\theta(G) - E_\theta(G_{-v})$, where G_{-v} is the graph obtained by removing v and all edges incident to it.

TABLE VI
ASR OF ATTACK METHODS AGAINST PRUNG IN THE 5 CROSS-CONTEXT SCENARIOS

Dataset	Model	Attack	Cross-Dist	Cross-Class	Cross-Dataset	Cross-Domain	Cross-Task
CiteSeer-N	ProG	GCBA_M	0.14	0.10	0.10	0.04	0.43
		GCBA_R	0.21	0.07	0.53	0.43	0.53
		TrojanGPL	0.92	0.92	0.97	0.93	0.95
	GraphPrompt	GCBA_M	0.67	0.54	0.69	0.32	0.66
		GCBA_R	0.57	0.14	0.70	0.33	0.54
		TrojanGPL	0.87	0.80	0.96	0.97	0.94
CiteSeer-G	ProG	GCBA_M	0.56	0.22	0.07	0.25	0.02
		GCBA_R	0.02	0.14	0.21	0.16	0.42
		TrojanGPL	0.92	0.90	0.97	0.96	0.93
	GraphPrompt	GCBA_M	0.53	0.38	0.49	0.52	0.50
		GCBA_R	0.56	0.37	0.55	0.50	0.53
		TrojanGPL	0.93	0.90	0.89	0.98	0.92

TABLE VII
ABLATION STUDY OF TROJANGPL AGAINST 2 GPL METHODS WITH PRUNG ON CITESEER ACROSS 5 CROSS-CONTEXT SCENARIOS

Attack	Cross Distribution		Cross Class		Cross Dataset		Cross Domain		Cross Task	
	ProG	GraphPrompt	ProG	GraphPrompt	ProG	GraphPrompt	ProG	GraphPrompt	ProG	GraphPrompt
w/o EA	0.90	0.74	0.87	0.02	0.93	0.03	0.93	0.02	0.89	0.10
w/o NFA	0.36	0.02	0.11	0.66	0.17	0.03	0.07	0.05	0.17	0.01
w/o TO	0.48	0.03	0.47	0.13	0.33	0.02	0.66	0.01	0.10	0.02
TrojanGPL	0.92	0.87	0.92	0.80	0.97	0.96	0.93	0.97	0.95	0.93

Proposition 3 (Node-Level Separability). Fix a downstream class c . There exist constants $\delta_c > 0$ and $\gamma > 0$ such that for graphs assigned to class c , the following holds. For any normal node $v \in V_{\text{clean}}$, $\|\Delta z_\theta(G, v)\| \leq \delta_c$, whereas for any trigger node $v \in V_{\text{trigger}}$, $\|\Delta z_\theta(G, v)\| \geq \delta_c + \gamma$.

Rationale. For clean graphs, node embeddings are optimized to encode semantic information consistent with the downstream class. As a result, the contribution of normal nodes to the aggregated graph embedding aligns with the class-specific representation structure, which constrains the magnitude of the embedding perturbation induced by node removal. In contrast, trigger nodes are optimized to activate a backdoor objective rather than preserve semantic coherence. Their contributions to the graph embedding are therefore disproportionately large and weakly aligned with class semantics. Removing such nodes leads to significantly larger representation perturbations, giving rise to the separation characterized in Proposition 3.

The node-level separability further induces a graph-level distinction when the most influential node is removed from each graph. Let $v^*(G) = \arg \max_{v \in V} \|\Delta z_\theta(G, v)\|$, and define the corresponding graph-level embedding shift as $\Delta E_\theta(G) = \Delta z_\theta(G, v^*(G))$.

Proposition 4 (Graph-Level Shift Consistency). Fix a downstream class c . For clean graphs G assigned to class c , the embedding shifts $\Delta E_\theta(G)$ exhibit consistent patterns in the embedding space. In contrast, for backdoored graphs assigned to the same class, the distribution of $\Delta E_\theta(G)$ deviates due to trigger-induced perturbations.

Rationale. Clean graphs predicted as the same class are optimized under a shared task objective, inducing a common semantic structure in the representation space. Removing the most influential node therefore perturbs embeddings along class-consistent directions, leading to similar representation shifts across clean samples. In contrast, backdoored graphs

contain an additional trigger-induced component that is misaligned with class semantics. Since $v^*(G)$ captures the node with the largest embedding influence, its removal suppresses this trigger-related component, resulting in representation shifts that differ markedly in both magnitude and direction from those of clean graphs.

Finally, the localized nature of trigger effects implies that iteratively removing nodes inducing anomalous representation shifts progressively suppresses trigger contributions. As trigger-related components are eliminated, the remaining embedding is increasingly dominated by class-consistent semantic representations. This explains why ES-Detector can both detect backdoored graphs via distributional discrepancies in representation shifts and mitigate backdoor effects through iterative node removal, while largely preserving performance on clean samples.

D. Evaluation of Our Defense

In this section, we present a comprehensive evaluation of our proposed *ES-Detector* defense framework using the experimental setup in Section IV-C1. Unless otherwise specified, we set $\epsilon = 1$ in all experiments. To evaluate defense effectiveness, we report two metrics: Main Task Accuracy (ACC), which measures the classification performance on clean samples after applying the defense; and Purified Attack Success Rate (PASR), which quantifies the residual attack success after the input has been processed by the defense.

1) *Baseline Defense Methodologies:* In cross-context GPL, the downstream defender typically has access only to a pre-trained GNN encoder and a small set of clean samples for prompt tuning, and does not perform encoder retraining. As a result, many backdoor defenses that require access to training or modifications to the GNN encoder are not directly applicable in this setting. For example, *RandSample* [6] and *GNNGuard* [26]

TABLE VIII
ACC / PASR OF TROJANGPL AGAINST VARIOUS DEFENSE METHODS

Defense	Cross Distribution			Cross Class			Cross Dataset		Cross Domain		Cross Task
	CiteSeer	Cora	Photo	CiteSeer	Cora	Photo	CiteSeer	Photo	CiteSeer	Cora	CiteSeer
Random-Node-Delete	0.74 / 0.82	0.72 / 0.69	0.78 / 0.99	0.92 / 1.00	0.87 / 0.65	0.76 / 0.99	0.82 / 0.81	0.81 / 0.99	0.82 / 0.97	0.72 / 0.98	0.55 / 0.68
Random-Edge-Delete	0.81 / 1.00	0.73 / 0.97	0.79 / 0.95	0.92 / 1.00	0.87 / 1.00	0.75 / 0.77	0.83 / 0.81	0.81 / 0.66	0.81 / 1.00	0.72 / 1.00	0.67 / 1.00
Embed-Noise	0.77 / 0.93	0.73 / 0.93	0.76 / 0.99	0.91 / 1.00	0.87 / 1.00	0.77 / 1.00	0.82 / 1.00	0.82 / 0.94	0.82 / 0.75	0.72 / 0.63	0.80 / 0.62
ES-Detector	0.80 / 0.22	0.75 / 0.17	0.77 / 0.13	0.92 / 0.00	0.87 / 0.00	0.76 / 0.20	0.77 / 0.21	0.81 / 0.15	0.83 / 0.15	0.71 / 0.13	0.81 / 0.15

TABLE IX
THE ACC/PASR OF TROJANGPL AGAINST ES-DETECTOR WITH DIFFERENT ϵ

Cross-context Scenarios	0.01	0.1	1
Cross Distribution	0.76 / 0.14	0.73 / 0.22	0.75 / 0.17
Cross Class	0.87 / 0.29	0.81 / 0.29	0.87 / 0.00
Cross Dataset	0.82 / 0.15	0.83 / 0.08	0.77 / 0.21
Cross Domain	0.71 / 0.14	0.72 / 0.19	0.71 / 0.13

involve training-time procedures or model-side changes, which are impractical under the cross-context GPL constraint. Therefore, we focus on lightweight defenses that operate purely at inference time without retraining or modifying the pretrained encoder. Following this principle, we consider several representative perturbation-based baselines.

Random-Node-Delete Defense: The defender randomly removes k nodes from the input graph together with their incident edges. This baseline aims to disrupt localized adversarial structures by applying random structural perturbations. Since the trigger subgraph in *TrojanGPL* contains three trigger nodes, we set $k = 3$ to use a comparable perturbation budget.

Random-Edge-Delete Defense: This defense randomly deletes k edges from the input graph while keeping all nodes intact. The motivation is that backdoor triggers may rely on specific connectivity patterns, which can be weakened by edge-level perturbations. We set $k = 3$ to match the perturbation budget.

Embedding-Noise Defense: This defense injects Gaussian noise into the *graph-level* embeddings before downstream classification. It is motivated by the intuition that trigger-induced embedding effects may be sensitive to stochastic perturbations. In our experiments, we add Gaussian noise with standard deviation 0.01 at inference time.

2) *Defense Performance Against TrojanGPL:* Table VIII shows that *ES-Detector* substantially reduces the attack success rate to 0.00–0.22 across challenging scenarios while preserving high clean accuracy. In contrast, the perturbation-based baselines (Random-Node-Delete, Random-Edge-Delete, and Embedding-Noise) yield much higher attack success rates, indicating limited robustness against *TrojanGPL*. Overall, these results suggest that *ES-Detector* can effectively mitigate backdoor triggers with minimal impact on benign performance.

The effectiveness of *ES-Detector* stems from detecting inconsistencies in embedding shifts induced by removing structurally influential nodes. Clean graphs exhibit stable embedding responses under such perturbations, whereas backdoored graphs show pronounced deviations due to trigger-related contributions. This discrepancy enables *ES-Detector* to identify suspicious inputs and suppress trigger effects via iterative node removal.

3) *Impact of MMD Threshold ϵ :* As shown in Table IX, ϵ governs the accuracy–security trade-off, and its impact is scenario-dependent. For cross-class attacks, $\epsilon = 1$ yields the lowest PASR while preserving ACC, whereas in cross-dataset settings, $\epsilon = 0.1$ achieves the strongest mitigation with the best ACC. In cross-distribution and cross-domain scenarios, smaller thresholds generally reduce PASR, though the trend is not fully consistent. Overall, these results suggest that a fixed ϵ may exhibit varying effectiveness across attack settings.

4) *Scalability Analysis of ES-Detector:* *ES-Detector* mainly incurs computation from repeated encoder forward passes during node-removal sensitivity analysis. For a prompted graph with $|V|$ nodes, identifying the most influential node requires evaluating $E_\theta(G_{-v})$ for all $v \in V$, resulting in a time complexity of $\mathcal{O}(|V|T_{\text{fwd}})$, where T_{fwd} denotes the cost of a single encoder forward pass. If at most K pruning iterations are performed, the overall complexity becomes $\mathcal{O}(K|V|T_{\text{fwd}})$, where K is a small constant in practice. Importantly, in GPL frameworks such as ProG, downstream node classification and link prediction tasks are reformulated as graph-level prediction tasks via prompted graph construction. As a result, *ES-Detector* operates on prompted graphs rather than the full raw network, making it computationally practical under the GPL pipeline, including for large datasets.

VI. CONCLUSION

This work investigates backdoor threats in graph prompt learning and identifies vulnerabilities introduced by cross-context knowledge transfer. We propose *TrojanGPL*, a task-agnostic attack that implants backdoors during pretraining by constructing trigger graphs and exploiting prompt-induced transformations. Theoretical and empirical results show that *TrojanGPL* achieves attack success rates above 90% across diverse settings while preserving clean-data performance. We further introduce *ES-Detector*, which detects backdoored samples through abnormal embedding shifts after influential-node removal and reduces attack success rates by at least 60% without prior trigger or downstream-task knowledge. These findings highlight the importance of representation-level robustness analysis for securing graph prompt learning.

REFERENCES

- [1] X. Sun, H. Cheng, J. Li, B. Liu, and J. Guan, “All in one: Multi-task prompting for graph neural networks,” in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2023, pp. 2120–2131.
- [2] Z. Liu, X. Yu, Y. Fang, and X. Zhang, “GraphPrompt: Unifying pre-training and downstream tasks for graph neural networks,” in *Proc. ACM Web Conf.*, 2023, pp. 417–428.
- [3] X. Sun, J. Zhang, X. Wu, H. Cheng, Y. Xiong, and J. Li, “Graph prompt learning: A comprehensive survey and beyond,” 2023, *arXiv:2311.16534*.

- [4] Z. Tan, R. Guo, K. Ding, and H. Liu, "Virtual node tuning for few-shot node classification," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2023, pp. 2177–2188.
- [5] T. Fang, Y. Zhang, Y. Yang, and C. Wang, "Prompt tuning for graph neural networks," 2022, *arXiv:2209.15240*.
- [6] Z. Zhang, J. Jia, B. Wang, and N. Z. Gong, "Backdoor attacks to graph neural networks," in *Proc. 26th ACM Symp. Access Control Models Technol.*, 2021, pp. 15–26.
- [7] J. Xu and S. Picek, "Poster: Clean-label backdoor attack on graph neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 3491–3493.
- [8] Y. Sheng, R. Chen, G. Cai, and L. Kuang, "Backdoor attack of graph neural networks based on subgraph trigger," in *Proc. Int. Conf. Collaborative Comput. Netw. Appl. Worksharing*, 2021, pp. 276–296.
- [9] K. Xu et al., "Topology attack and defense for graph neural networks: An optimization perspective," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 3961–3967.
- [10] Z. Xi, R. Pang, S. Ji, and T. Wang, "Graph backdoor," in *Proc. USENIX Secur. Symp.*, 2021, pp. 1523–1540.
- [11] X. Dong et al., "Adaptive backdoor attacks with reasonable constraints on graph neural networks," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 4053–4069, Jul./Aug. 2025.
- [12] X. Lyu, Y. Han, W. Wang, H. Qian, I. Tsang, and X. Zhang, "Cross-context backdoor attacks against graph prompt learning," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 2094–2105.
- [13] Y. Yan, P. Zhang, Z. Fang, and Q. Long, "Inductive graph alignment prompt: Bridging the gap between graph pre-training and inductive fine-tuning from spectral perspective," in *Proc. ACM Web Conf.*, 2024, pp. 4328–4339.
- [14] X. Yu, C. Zhou, Y. Fang, and X. Zhang, "MultiGPrompt for multi-task pre-training and prompting on graphs," in *Proc. ACM Web Conf.*, 2024, pp. 515–526.
- [15] X. Yu, Y. Fang, Z. Liu, and X. Zhang, "HGPrompt: Bridging homogeneous and heterogeneous graphs for few-shot prompt learning," in *Proc. AAAI Conf. Artif. Intell.*, 2024, Art. no. 1848.
- [16] E. Dai, M. Lin, X. Zhang, and S. Wang, "Unnoticeable backdoor attacks on graph neural networks," in *Proc. ACM Web Conf.*, 2023, pp. 2263–2273.
- [17] H. Xia, X. Zhao, R. Zhang, S. Xu, and L. Wang, "Clean-label graph backdoor attack in the node classification task," in *Proc. AAAI Conf. Artif. Intell.*, 2025, Art. no. 2412.
- [18] Y. Ding, Y. Liu, Y. Ji, W. Wen, Q. He, and X. Ao, "SPEAR: A structure-preserving manipulation method for graph backdoor attacks," in *Proc. ACM Web Conf.*, 2025, pp. 1237–1247.
- [19] H. Zhang, J. Chen, L. Lin, J. Jia, and D. Wu, "Graph contrastive backdoor attacks," in *Proc. Int. Conf. Mach. Learn.*, 2023, Art. no. 1714.
- [20] Z. Yang, W. Cohen, and R. Salakhudinov, "Revisiting semi-supervised learning with graph embeddings," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 40–48.
- [21] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," 2018, *arXiv:1811.05868*.
- [22] K. M. Borgwardt, C. S. Ong, S. Schöner, S. Vishwanathan, A. J. Smola, and H.-P. Kriegel, "Protein function prediction via graph kernels," *Bioinformatics*, vol. 21, no. suppl_1, pp. i47–i56, 2005.
- [23] P. Velickovic et al., "Graph attention networks," *Stat*, vol. 1050, no. 20, 2017, Art. no. 48550.
- [24] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, "Masked label prediction: Unified message passing model for semi-supervised classification," 2020, *arXiv:2009.03509*.
- [25] H. Wu, C. Wang, Y. Tyshetskiy, A. Docherty, K. Lu, and L. Zhu, "Adversarial examples on graph data: Deep insights into attack and defense," 2019, *arXiv:1903.01610*.
- [26] X. Zhang and M. Zitnik, "GNNGUARD: Defending graph neural networks against adversarial attacks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 777.



Xiaoting Lyu received the PhD degree from Beijing Jiaotong University, in 2025. She is the assistant professor with the Ministry of Education Key Lab for Intelligent Networks and Network Security, aka MOE KLINNS Lab, Xi'an Jiaotong University, China.



Yufei Han is a senior researcher with INRIA France. His research interests encompass trustworthy and interpretable AI technologies for cybersecurity applications, as well as the adversarial robustness of AI systems. He has published more than 50 research papers on top-tiered venues on AI and cybersecurity, like IEEE S&P, CCS, KDD, NDSS and AAAI. He regularly serves as program committees and peer reviews in these venues.



Hangwei Qian received the BEng degree from the University of Science and Technology of China (USTC), in 2015, and the PhD degree in computer science and engineering from Nanyang Technological University (NTU), Singapore, in 2020. She is currently a scientist with Centre for Frontier AI Research (CFAR), A*STAR Research Entities, Singapore. Previously, she was awarded the prestigious Wallenberg-NTU Presidential Postdoctoral Fellowship between 2020 and 2022. Her research interests lie in trustworthy AI, transfer learning, and AI for science. She has published top-tier papers on KDD, IJCAI, AAAI, and AIJ over the years.



Ivor W. Tsang (Fellow, IEEE) has been the director with A*STAR Centre for Frontier AI Research (CFAR) since 2022. He is working with the forefront of Big Data analytics and Artificial Intelligence. His research interests include transfer learning, deep generative models, learning with weakly supervision, and Big Data analytics. His work is recognised internationally for its outstanding contributions to those fields. In 2013, he was the recipient of his ARC Future Fellowship for his outstanding research on Big Data analytics and large-scale machine learning. He was conferred the IEEE fellow for his outstanding contributions to large-scale machine learning and transfer learning. He is the Editorial Board of the *Journal of Machine Learning Research*, *Machine Learning*, *Journal of Artificial Intelligence Research*, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, and *IEEE Transactions on Artificial Intelligence*. He is a senior area chair/area chair for NeurIPS, ICML, AAAI, and IJCAI, and the steering committee of ACML.



Wei Wang received the PhD degree from Xi'an Jiaotong University, in 2006. He is a full professor with the Ministry of Education Key Lab for Intelligent Networks and Network Security, aka MOE KLINNS Lab, Xi'an Jiaotong University, China. He is also an adjunct professor with the School of Computer Science and Technology, Beijing Jiaotong University. He was a post-doctoral researcher with the University of Trento, Italy, from 2005 to 2006. He was a post-doctoral researcher with TELECOM Bretagne and with INRIA, France, from 2007 to 2008. He was also a European ERCIM fellow with the Norwegian University of Science and Technology (NTNU), Norway, and with the Interdisciplinary Centre for Security, Reliability, and Trust (SnT), University of Luxembourg, from 2009 to 2011. He was a faculty member with Beijing Jiaotong University from 2011 to 2024. His recent research interests lie in privacy-preserving computation and blockchain. He has authored or co-authored more than 100 peer-reviewed articles in various journals and international conferences, including *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Software Engineering*, *USENIX Security*, *ACM CCS*, *AAAI*, *Ubicomp*, *IEEE INFOCOM*. He received the ACM CCS 2023 Distinguished Paper Award. He is an Elsevier "highly cited Chinese Researchers". He is an associate editor of the *IEEE Transactions on Dependable and Secure Computing*, and an Editorial Board member of the *Computers & Security* and *Frontiers of Computer Science*. He is a vice chair of ACM SIGSAC China.